SurplusSense Individual Report

Project: SurplusSense **GitHub Repository:** https://github.com/TohzKai/surplusSense **Submission Type:** MGMT655 Individual Assignment **Product Type:** ML decision-support cockpit for food surplus management

Private repository: https://github.com/TohzKai/surplusSense

Prof. Jack Hong has been invited as a collaborator for grading. The repository will remain private and access will be removed after grading, as instructed.

---

1. Executive Summary for Business Manager

What SurplusSense Does

SurplusSense is a merchant-side food surplus decision-support cockpit. It predicts how much surplus an F&B outlet will have at end of day and recommends what action to take — hold, monitor, discount, donate, or discard — before unsold inventory becomes waste.

The product combines XGBoost ML surplus prediction with transparent rule-based recommendation logic, food-safety gating, and revenue recovery estimation, presented in an interactive Streamlit interface.

Who It Is For

Bakeries, cafés, and small F&B operators with 5–50 daily surplus units. Singapore-based, SFA-licensed food operators. The merchant or floor manager standing at the counter at 3–4pm is the primary user.

Why Food Surplus Is a Business Problem

Singapore's F&B sector generates approximately 784,000 tonnes of food waste annually (NEA, 2024). Food businesses bear the full cost of surplus — ingredient, labour, and disposal — with zero recovery when discarded. The average F&B outlet loses SGD 200–500 per month to avoidable food waste.

Why the App Is Ready for Controlled Pilot Launch

The ML pipeline is trained, validated, and reproducible. The decision workflow is implemented end-to-end. The model demonstrates 57% improvement over a historical-average baseline on temporal holdout validation. 75 unit tests pass. The product is **pilot-ready**, not production-proven.

Main Business Value

1. **Reduce avoidable food waste** — actionable recommendations before items become waste
2. **Improve recovery value** — discount or donate items instead of discarding
3. **Support more consistent manager decisions** — structured recommendation replaces ad-hoc judgment

Important Caveat

All current results are from synthetic data. Real merchant data pilot is required before commercial deployment claims.

---

2. Product Walkthrough for App User

Who Uses the App

Bakery or café operators, floor managers, or outlet owners at 3–4pm when deciding what to do with unsold inventory. No ML knowledge required.

What the User Selects

The sidebar allows the user to set:

- **Merchant** — outlet name and type (Bakery, Café, Small F&B)
- **Category** — product category (Bread, Pastries, Cakes, etc.)
- **Product** — specific item name
- **Date** — the operating day
- **Time** — current time for safety window calculations
- **Cold-start mode** — for new merchants with fewer than 30 days of history

What Outputs the App Gives

1. **Predicted surplus quantity** — estimated units that will go unsold today
2. **Recommended action** — HOLD / MONITOR / DISCOUNT / DEEP DISCOUNT / DONATE / DISCARD
3. **Recommended discount tier** — 20–70% off
4. **Food safety status** — SAFE / CAUTION / BLOCK
5. **Revenue recovery estimate** — SGD amount likely to be recovered
6. **Listing schedule** — when to list and pickup deadline

How to Interpret Recommendations

Recommendation	Meaning	When to use
HOLD	No action needed yet	Surplus is low; check again at 5pm
MONITOR	Watch for 1–2 hours	Moderate surplus; reassess before discounting
DISCOUNT	Apply 20–40% off	Moderate surplus with adequate shelf life
DEEP DISCOUNT	Apply 50–70% off	High surplus or short shelf life remaining
DONATE	Redirect to charity	Item safe but no consumer demand
DISCARD	Dispose safely	Item failed safety check

Important Cautions

- The app is **decision support, not automated decision-making** — the user always decides
- Safety checks are advisory; the user remains responsible for food safety compliance
- Predictions are estimates from synthetic data; real merchant results will differ

- Cold-start predictions (new merchants) use category averages and are less accurate

---

3. ML and Decision-Support Approach

The ML Technique

SurplusSense uses **supervised machine learning** — specifically **XGBoost regression** — to predict surplus units per item per day. This is a structured regression problem: the model learns a mapping from historical patterns to an expected surplus quantity.

The Features

The model uses **31 raw feature fields, expanded into 47 model input columns after one-hot encoding** for categorical variables. Feature groups include:

- **Temporal**: day of week, weekend flag, month, cyclical encodings
- **Lag**: previous day surplus, same weekday last week surplus
- **Rolling**: 7-day average, max, and std of surplus
- **Expanding-window aggregate**: merchant, category, and day-of-week historical averages (computed using only past data — no future leakage)
- **Product**: price, shelf life, preparation time, storage type
- **Derived**: holding-time-to-shelf-life ratio, weekend-promotion interaction

ML Plus Business Rules

ML prediction alone tells the merchant "expected waste = 8 units." Business rules convert this into "apply 40% discount, list by 5pm, expect SGD 32 recovery." This is why SurplusSense is a decision cockpit, not an ML dashboard.

The final recommendation flow is:

1. Predict surplus (XGBoost)
2. Assess shelf life and expiry pressure (safety rules)
3. Estimate revenue recovery (discount calculation)
4. Recommend action (rule engine)
5. Explain the recommendation (merchant-readable reasoning)

Why ML Plus Rules Beats Raw Prediction Alone

- Food safety cannot be entrusted to a statistical model — rules provide deterministic safety guarantees
- Merchant trust requires explainability — every recommendation traces to a specific rule
- No feedback loop required — rules do not need consumer purchase data to improve

---

4. Validation and Results

Validation Method

Temporal 80/20 holdout — the last 20% of dates (sorted chronologically) form the test set. This is the primary evaluation method because it simulates how the model would actually be deployed: predicting future surplus from past data, not retrodicting randomly mixed historical records.

Official Metrics

Metric	Value
XGBoost MAE (temporal holdout)	**0.6355**
XGBoost RMSE (temporal holdout)	0.8131
XGBoost R^2 (temporal holdout)	0.9068
Historical Average baseline MAE	1.49
Improvement vs baseline	**57.2%**

Official metric source: `outputs/model_comparison.csv`

Unit Tests

```
python -m pytest tests/unit/ -q
75 passed
```

Evaluation Command

```
python src/evaluate_model.py
```

Displays official temporal holdout metrics from `outputs/model_comparison.csv`.

Diagnostic Evaluation

A broader-dataset evaluation is available with `python src/evaluate_model.py --diagnostic`, producing MAE ~0.47. This is **not** the official metric — it evaluates on the full engineered dataset rather than the forward-looking holdout period and can overstate performance.

Data Limitation

Current results are from **synthetic data** generated by `data/generate_synthetic_data.py`. Real merchant data pilot is required before production deployment claims.

---

5. Launch Readiness and Business Case

Target Users

Singapore-based bakeries, cafés, and small F&B operators with:

- 5–50 daily surplus units
- SGD 5–15 average unit price
- Structured end-of-day surplus decision workflow
- Digital readiness for app access

Excluded from MVP: Hawker stalls (collective family decision-making), large chains (existing waste systems).

Recommended Launch Path

Controlled pilot with 3–5 volunteer outlets over 4 weeks:

- Week 1–2: App setup, staff training, baseline establishment
- Week 3–4: Active recommendation phase with daily feedback logging

Pilot Success Metrics

Metric	Target
Recommendation acceptance rate	> 60%
Revenue recovery	> SGD 200/outlet/month
Food safety incidents	0
Manager NPS	> 40

Key Risks and Mitigations

Risk	Mitigation
Merchant over-reliance on ML	Human-in-the-loop framing; app says "recommends," merchant decides
Safety misfire	BLOCK/CAUTION/SAFE gate overrides commercial optimization
Model degrades over time	Phase 2: quarterly retraining pipeline
Data quality poor	Pilot data-validation phase before full deployment
Overclaiming from synthetic data	Explicit disclosure: pilot-ready, not production-proven

Commercial Logic

SaaS pricing proposed at SGD 99–299/month per outlet. At SGD 200/month recovered and SGD 99/month cost, the product pays for itself with SGD 101 net monthly benefit. Transaction fee (15% of recovered value) builds as volume grows.

---

6. Developer Handover

Key Files

File	Purpose
app/streamlit_app.py	Main app — Streamlit decision cockpit
src/train_model.py	XGBoost training pipeline
src/evaluate_model.py	Official temporal holdout metric display
src/feature_engineering.py	Feature engineering (31 raw → 47 model columns)
src/recommendation_engine.py	10-tier discount engine + cold-start fallback
src/food_safety_rules.py	5 safety checks: BLOCK/CAUTION/SAFE
outputs/model_comparison.csv	Official temporal holdout metrics
COC_DECISION_LOG.md	Decision journey and judgment evidence
docs/DEVELOPER_HANDOVER.md	Full developer guide
docs/USER_GUIDE.md	End-user guide
FINAL_SUBMISSION_PACKAGE.md	Submission overview

Run Commands

```
pip install -r requirements.txt
streamlit run app/streamlit_app.py
```

Test and Evaluation

```
python -m pytest tests/unit/ -q   # 75 passed
python src/evaluate_model.py         # Official temporal holdout metrics
```

Future Developer Priorities

1. Replace synthetic data with real merchant data through pilot partnerships
2. Add POS/inventory integration for automated item data entry
3. Validate with outlet managers; measure actual vs. predicted surplus
4. Build quarterly retraining pipeline for model drift monitoring
5. Add multi-tenant authentication if scaling beyond single outlet

---

7. COC Decision Summary

From Prototype to Pilot-Ready Product

The weekly prototype predicted surplus with ML. The mature product converts that prediction into a merchant action — with safety gates, recovery estimates, and explainable recommendations.

Key Decisions

Decision	Choice	Why
Target user	Merchant operations	Single-sided acquisition, clearer pain than consumer marketplace
Problem scope	Decision support	Faster to deploy, clearer value than full marketplace
Model role	Supporting layer	Prediction alone does not help merchant act
Recommendation logic	Rule-based	Transparent, explainable, no feedback loop needed
Food-safety logic	Active gate	BLOCK/CAUTION/SAFE gating; safety non-negotiable
Cold-start	Category benchmark	Immediate value for new merchants
Validation method	Temporal holdout	Prevents future data leakage; simulates real deployment

Rejected Alternatives

Rejected	Why
Consumer marketplace	Chicken-and-egg problem; direct competition with established players
ML dashboard (prediction as output)	Answers "what happened?" not "what do I do?"
Black-box learned policy	Requires feedback loop; unsafe for food-safety decisions
LLM for recommendations	Non-deterministic; potential hallucinations in safety contexts
Overclaiming production readiness	Synthetic data only; pilot validation required

Evidence of Judgment

Every significant product decision — from rejecting consumer marketplace to adding safety gates — was made by deliberate evaluation of trade-offs. 75 passing unit tests, 57% holdout improvement, 5 safety checks, and 10-tier recommendation engine are all outputs of human judgment, not AI defaults.

---

8. Submission Note

Private repository: https://github.com/TohzKai/surplusSense

Prof. Jack Hong has been invited as a collaborator for grading. The repository will remain private and access will be removed after grading, as instructed.

Supporting Files

File	Description
COC_DECISION_LOG.md	Prototype-to-product decision journey
docs/DEVELOPER_HANDOVER.md	Full technical documentation
docs/USER_GUIDE.md	End-user walkthrough
FINAL_SUBMISSION_PACKAGE.md	Submission overview
app/streamlit_app.py	Working interactive product

ML Technique Family

SurplusSense uses **supervised regression with XGBoost** — predicting surplus units as a continuous value. This differs from the team project, WanderLess, which uses **recommender systems and optimization** (hybrid tourist-guide matching, TruncatedSVD collaborative filtering, and itinerary optimization).

SurplusSense is a pilot-ready decision-support prototype. All performance metrics are from synthetic data and require real merchant validation before commercial deployment claims.